PROJEKT SYKOM

Albert Kołsut

8 marca 2026

Spis treści

1. Tematyka zadania projektowego 1
2. Zadanie do wykonania 1
3. Moduł w języku Verilog 2
4. Testy 2
4.1. Testy Verilog 2
4.2. Test Kernel 6
4.3. Test main.c 7

1. Tematyka zadania projektowego

Niniejsze sprawozdanie dokumentuje przebieg realizacji projektu wykonanego w ramach przedmiotu SYKOM. Celem projektu było utworzenie układu SoC z peryferiami wytworzonymi przez siebie i emulowanymi przez spersonalizowany program QEMU oraz testowanie tego systemu z wykorzystaniem tworzonej dla tego środowiska dystrybucji systemu Linux i odpowiednich sprzętowi sterowników systemowych.

2. Zadanie do wykonania

W ramach projektu z przedmiotu SYKOM otrzymałem zadanie polegające na zaprojektowaniu modułu sprzętowego w języku Verilog, którego celem było obliczanie sumy kontrolnej typu **CRC16-LJ1200** co daje wilomian **POLY = 0x6F63** na danych przechowywanych w wewnętrznym bloku pamięci o rozmiarze 250 bajtów. Dane miały być wprowadzane pojedynczo (po 8 bitów) przez rejestr wejściowy IN **adres 0x560**, a sterowanie pracą modułu odbywało się za pomocą rejestru CTRL **adres 0x578**. Stan działania modułu udostępniany był przez rejestr STATE **adres 0x568**, a wynik obliczeń przez rejestr RESULT **adres 0x570**. Dodatkowo rejestr CTRL ma w zaprojektowanym przeze mnie module cztery funkcje:

— `CTRL_NOP` – "no operation", nie robiący nic, wartość 0
— `CTRL_CLEAR` – odpowiada za czyszczenie bufora, wartość 1
— `CTRL_RESET_CUR` – odpowiada za resetowanie indeksu bufora, wartość 2
— `CTRL_CALC_CRC` – odpowiada za rozpoczęcie liczenia CRC, wartość 3

Kolejnym elementem projektu było przygotowanie modułu jądra systemu Linux, który udostępniał powyższe rejestry w systemie plików PROCFS jako pliki tekstowe w formacie HEX:

— dskoal (IN),
— dtkoal (CTRL),
— dckoal (STATE),
— drkoal (RESULT).

Aby zademonstrować poprawność działania całego systemu, wymagana była również implementacja aplikacji użytkownika w języku C, wbudowanej w system plików (rootfs.ext2), która automatycznie przeprowadzała różne scenariusze testowe. Jej zadaniem było sprawdzenie funkcjonalności zaprojektowanego modułu i interfejsu poprzez interakcję z plikami PROCFS.

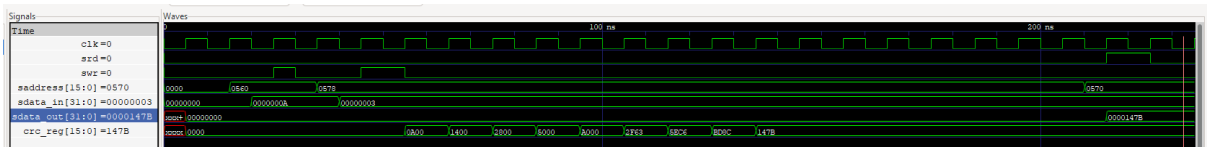
3. Moduł w języku Verilog

Zgodnie z otrzymanym poleceniem, utworzony moduł gpioemu.v ma za zadanie przyjmować pojedyńczo dane o wielkości 8 bitów poprzez rejestr wejściowy IN a następnie w momencie wysłania przez użytkowanika 3 pod adres 0x578 moduł przechodzi ze STATE IDLE (0), czyli podstawowego stanu, do stanu `PROCESS_BYTE` (1) a następnie do `PROCESS_BIT`, gdzie przechodzi przez kolejne bity wprowadzonej danej, gdy dojdzie do ostatniego bitu a w kolejce są jeszcze dane przechodzi do następnego bajtu (1) i zaczyna przechodzić przez kolejne bity (2). Po zakończeniu liczenia sumy kontrolnej moduł STATE przywracany jest do IDLE (0). Istnieje możliwość odczytania wyniku, który jest dostępny dla procesora poprzez adres 0x570. W takim wypadku z chwilowej wartośći CRC `CRC_reg` zostanie przekazana wartość na `sdata_out_s`. Moduł ma dodatkowe funkcje takie jak reset buffora, który zeruje wprowadzone dane i pozwala na wprowaqdzenie np. innych danych pozbywając się wszystkich poprzednich. Drugą opcją jest reset kursora który zostawia poprzednio wprowadzone dane ale pozwala je nadpisywać np po wprowadzeniu danych 0xAA oraz 0xBB zresetowaniu kursora i wprowadzeniu 0xDD, dane, z których będzie liczone CRC, to 0xDD 0xBB.

4. Testy

4.1. Testy Verilog

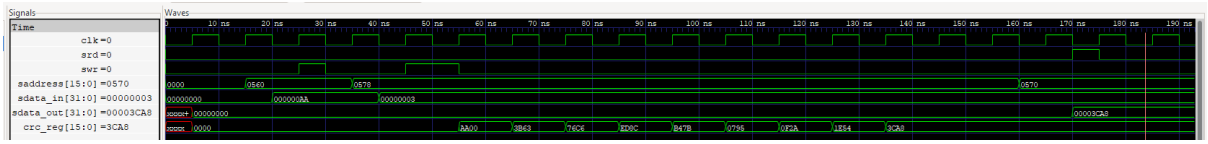
TEST 1



Rys. 1. Test przeliczania CRC dla 0A

Wprowadzamy 0x0A na adres 0x560, następnie dajemy polecenie rozpoczęcia liczenia CRC - 3 na adres 0x578, moduł zaczyna przeliczać CRC. Kolejne wartości chwilowe widać na `crc_reg` po zakiończeniu obliczeń wynik odczytujemy poprzez rejestr RESULT adres 0x570. Otrzymany wynik zgadza się z zakładanym, czyli **0x147B**.

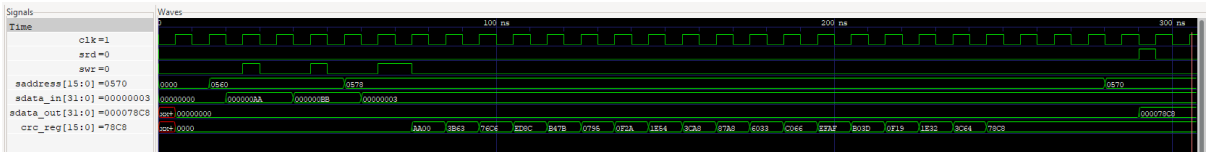
TEST 2



Rys. 2. Test przeliczania CRC dla AA

Wprowadzamy 0xAA na adres 0x560, następnie dajemy polecenie rozpoczęcia liczenia CRC - 3 na adres 0x578, moduł zaczyna przeliczać CRC. Kolejne wartości chwilowe widać na `crc_reg` po zakiończeniu obliczeń wynik odczytujemy poprzez rejestr RESULT adres 0x570. Otrzymany wynik zgadza się z zakładanym, czyli **0x3CA8**.

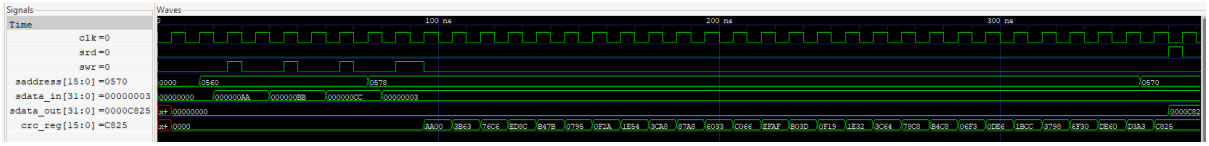
TEST 3



Rys. 3. Test przeliczania CRC dla AA BB

Wprowadzamy 0xAA 0xBB na adres 0x560, następnie dajemy polecenie rozpoczęcia liczenia CRC - 3 na adres 0x578, moduł zaczyna przeliczać CRC. Kolejne wartości chwilowe widać na `crc_reg` po zakiończzeniu obliczeń wynik odczytujemy poprzez rejestr RESULT adres 0x570. Otrzymany wynik zgadza się z zakładanym, czyli **0x78C8**.

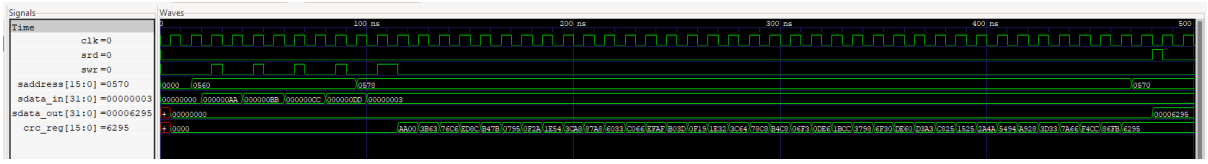
TEST 4



Rys. 4. Test przeliczania CRC dla AA BB CC

Wprowadzamy 0xAA 0xBB 0xCC na adres 0x560, następnie dajemy polecenie rozpoczęcia liczenia CRC - 3 na adres 0x578, moduł zaczyna przeliczać CRC. Kolejne wartości chwilowe widać na `crc_reg` po zakiończzeniu obliczeń wynik odczytujemy poprzez rejestr RESULT adres 0x570. Otrzymany wynik zgadza się z zakładanym, czyli **0xC825**.

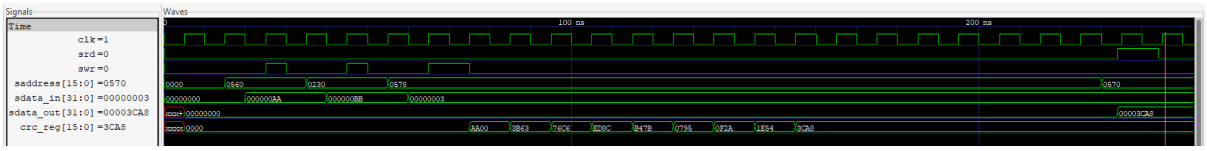
TEST 5



Rys. 5. Test przeliczania CRC dla AA BB CC DD

Wprowadzamy 0xAA 0xBB 0xCC 0xDD na adres 0x560, następnie dajemy polecenie rozpoczęcia liczenia CRC - 3 na adres 0x578, moduł zaczyna przeliczać CRC. Kolejne wartości chwilowe widać na `crc_reg` po zakiończzeniu obliczeń wynik odczytujemy poprzez rejestr RESULT adres 0x570. Otrzymany wynik zgadza się z zakładanym, czyli **0x6295**.

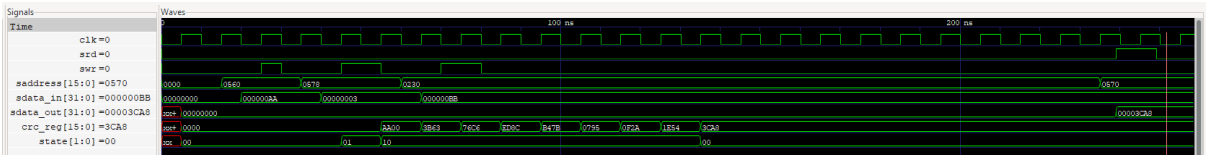
TEST 6



Rys. 6. Test na neutralność - odporność na nieprawidłowe adresy

Wprowadzamy 0xAA na adres 0x560, następnie wprowadzamy dane 0xBB pod błedny adres 0x230, następnie dajemy polecenie rozpoczęcia liczenia CRC - 3 na adres 0x578, moduł zaczyna przeliczać CRC. Kolejne wartości chwilowe widać na `crc_reg` po zakiończzeniu obliczeń wynik odczytujemy poprzez rejestr RESULT adres 0x570. Otrzymany wynik powinien być taki jak dla **TEST 2**, w którym to liczyliśmy CRC dla danych 0xAA. Wynik zgadza się z zakładanym, czyli **0x3CA8**.

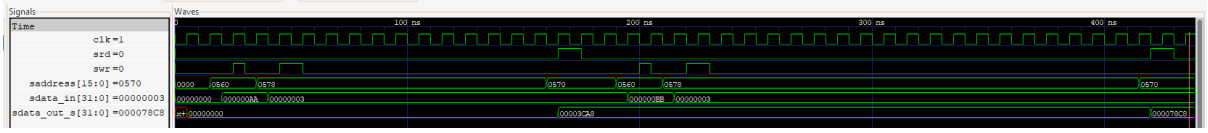
TEST 7



Rys. 7. Test na neutralność - odporność na nieprawidłowe adresy

Tym razem błedny adres został podany po rozpoczęciu liczenia sumy kontrolnej. Jak widać na zdjęciu numer 7 moduł ignoruje niepoprawny adres i nie zmienia ani wyniku ani stanu w jakim się znajduje. Otrzymany wynik zgadza się z zakładanym, czyli **0x3CA8**

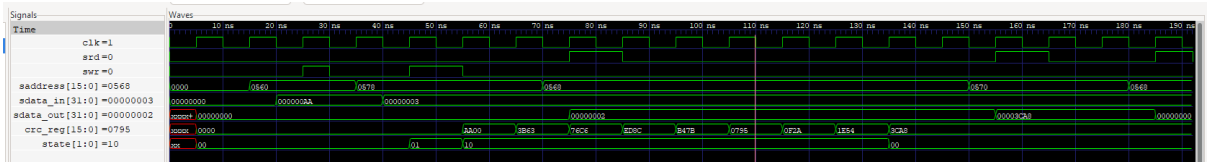
TEST 8



Rys. 8. Test przeliczania dwóch CRC jeden po drugim

Wprowadzamy 0xAA na adres 0x560, następnie dajemy polecenie rozpoczęcia liczenia CRC - 3 na adres 0x578, moduł zaczyna przeliczać CRC. Po zakiończzeniu obliczeń odczytujemy wynik. Wprowadzamy kolejne dane, w tym wypadku 0xBB na adres 0x560 i ponowanie rozpoczynamy liczenie CRC. Na koniec odczytujemy wynik. Widać, że otrzymana suma kontrolna jest taka sama jak dla **TEST3**, czyli dla danych 0xAA 0xBB, co oznacza, że otrzymany wynik zgadza się z zakładanym **0x78C8**.

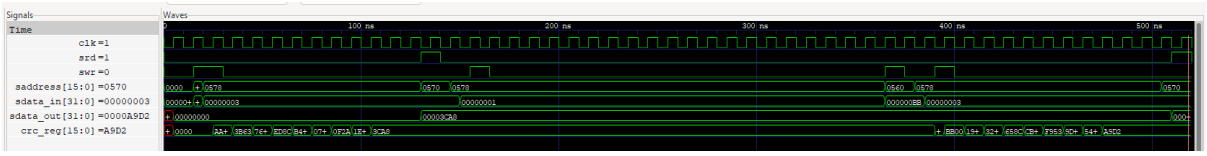
TEST 9



Rys. 9. Test odczytu STATE

W momencie, w którym wprowadzamy sygnał odczytu srd na 1 na `sdata_out` pojawia się 2, co odpowiada aktualnej wartości STATE, co świadczy o tym, że rejestr STATE adres 0x568 działa poprawnie. Dodawtkowo po zakończeniu obliczeń ponawiamy próbe odczytu stanu i zgadza się on z aktualną wartością czyli 0.

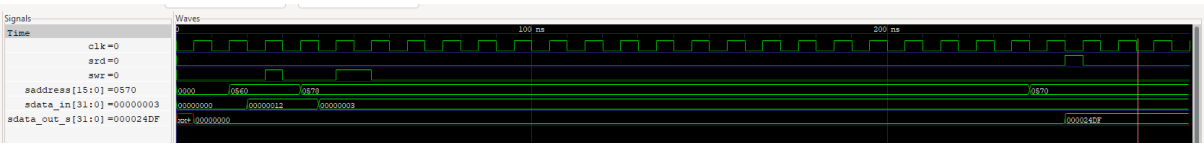
TEST 10



Rys. 10. Test czyszczenia buffra

Wprowadzamy 0xAA na adres 0x560, następnie dajemy polecenie rozpoczęcia liczenia CRC - 3 na adres 0x578, moduł zaczyna przeliczać CRC. Po zakiończzeniu obliczeń wynik odczytujemy poprzez rejestr RESULT, następnie rozpoczynamy czyszczenie buffora poprzez wprowadzenie 1 na adres 0x578 i czekamy aż operacja się zakończy. Wpowadzamy dane 0xBB zaczynamy liczenie CRC i odczytujemy wynik. Otrzymaliśmy **0xA9D2**, co odpowiada sumie kontrolen z 0xBB dla mojego CRC16, czyli wynik zgadza się z zakładanym.

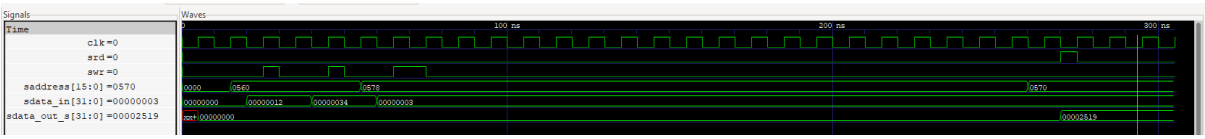
TEST 11



Rys. 11. Test przeliczania CRC dla 12

Wprowadzamy 0x12 i rozpoczynamy obliczanie CRC następnie odczytujemy wynik. Otrzymany wynik zgadza się z zakładanym i wynosi**0x24DF**.

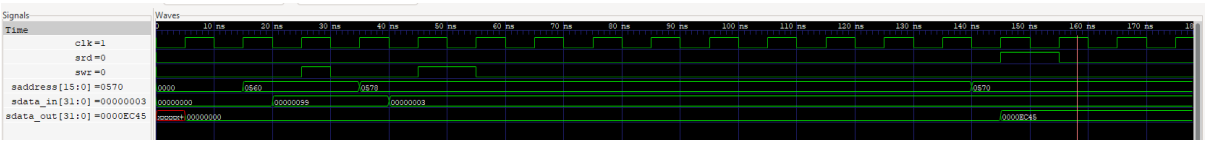
TEST 12



Rys. 12. Test przeliczania CRC dla 12 34

Wprowadzamy 0x12 0x34 i rozpoczynamy obliczanie CRC następnie odczytujemy wynik. Otrzymany wynik zgadza się z zakładanym i wynosi**0x2519**.

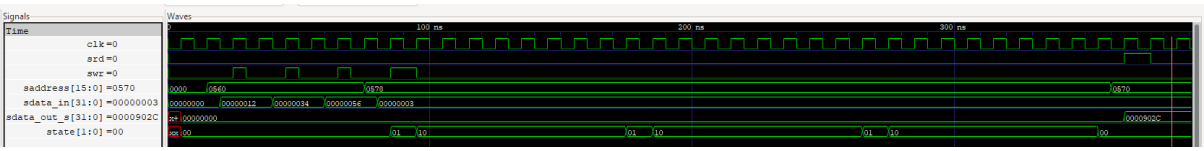
TEST 13



Rys. 13. Test przeliczania CRC dla 99

Wprowadzamy 0x99 i rozpoczynamy obliczanie CRC następnie odczytujemy wynik. Otrzymany wynik zgadza się z zakładanym i wynosi**0xEC45**

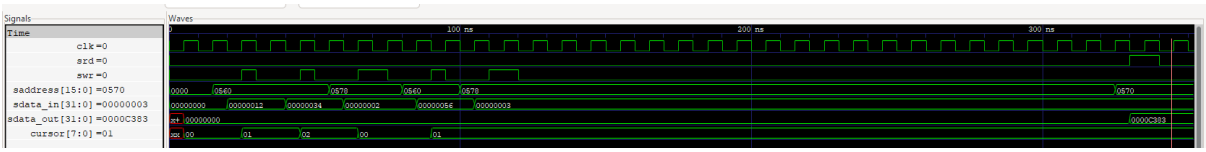
TEST 14



Rys. 14. Test przeliczania CRC dla 12 34 56 z widocznymi przejściami stanu

Wprowadzamy 0x12 0x34 0x56 i rozpoczynamy obliczanie CRC następnie odczytujemy wynik. Otrzymany wynik zgadza się z zakładanym i wynosi**0x902C**. Dodatkowo możemy zaobserwować na pozycji STATE jak zmieniają się stany podczas pracy modułu.

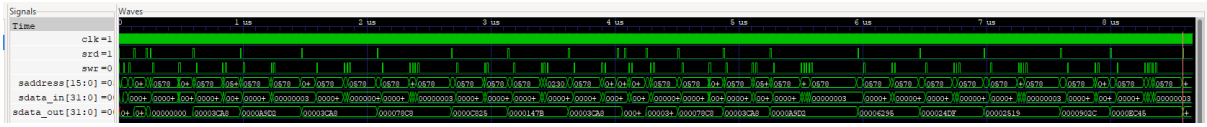
TEST 15



Rys. 15. Test resetu kursora z widocznym wskaźnikiem i wyliczenie CRC dla 56 34

Wprowadzamy 0x12 0x34 następnie poprzez rejestr CTRL wydajemy polecenie resetu kursora "2" wprowadzamy 0x56, które wskakuje na miejsce 0x12, zaczynamy liczyć CRC i odczytujemy wynik. Otrzymany wynik zgadza się z zakładanym i wynosi **0xC383**

WSZYSTKIE TESTY



Rys. 16. Przebiegi wszystkich testów

4.2. Test Kernel

Jest to dodatkowe potwierdzenia działania projektu, gdyż właściwa weryfikacja zostanie pokazana za pomocą aplikacji main.c.

TEST

```
# echo AA > /proc/dskoal
# echo 3 > /proc/dtkoal
# cat /proc/drkoal
00003CA8
# echo BB > /proc/dskoal
# echo 3 > /proc/dtkoal
# cat /proc/drkoal
000078C8
# echo CC > /proc/dskoal
# echo 3 > /proc/dtkoal
# cat /proc/drkoal
0000C825
# echo DD > /proc/dskoal
# echo 3 > /proc/dtkoal
# cat /proc/drkoal
00006295
```

Rys. 17. Kolejne wartości CRC dla danych wejściowych 0xAA, 0xAA 0xBB, 0xAA 0xBB 0xCC, 0xAA 0xBB 0xCC 0xDD

```
# echo 0A > /proc/dskoal
# echo 3 > /proc/dtkoal
# cat /proc/drkoal
0000147B
# echo 1 > /proc/dtkoal
# echo 3 > /proc/dtkoal
# cat /proc/drkoal
00000000
# echo AA > /proc/dskoal
# echo BB > /proc/dskoal
# echo CC > /proc/dskoal
# echo 3 > /proc/dtkoal
# cat /proc/drkoal
0000C825
```

Rys. 18. Wartośc CRC dla 0x0A test wyzerowania kursora i obliczenie CRC dla 0xAA 0xBB 0xCC

4.3. Test main.c

W celu przetestowania poprawności całego systemu została użyta aplikacja testująca stworzona w pliku main.c. Po uruchomieniu przez użytkownika automatycznie demonstruje działanie systemu w różnych sceenariuszach testowych.

```
# ./main
[TEST 1] CRC dla 0x0A
 CRC = 0x0000147B

[TEST 2] CRC dla 0xA0
 CRC = 0x000028D3

[TEST 3] CRC dla 0xAA
 CRC   = 0x00003CA8

[TEST 4] CRC dla 0xAA 0xBB
 CRC = 0x000078C8

[TEST 5] CRC dla 0xAA 0xBB 0xCC
 CRC = 0x0000C825

[TEST 6] CRC dla 0xAA 0xBB 0xCC 0xDD
 CRC = 0x00006295

[TEST 7] Same stany STATE dla AA
 STATE = 1
 STATE = 2
 STATE = 0
[TEST 8]  BB, czyszczenie, AA, CRC
 CRC = 0x00003CA8

[TEST 9] 12, 34, reset kursora, 56, CRC
 CRC = 0x0000C383

[TEST 10] Dwa CRC jeden po drugim: AA potem BB
 CRC after AA = 0x00003CA8
 CRC after BB = 0x000078C8

[TEST 11] CRC dla 0x12
 CRC = 0x000024DF

[TEST 12] CRC dla 0x12 0x34
 CRC = 0x00002519

[TEST 13] CRC dla 0x12 0x34 0x56
 CRC = 0x0000992C
[   18.487975] random: fast init done

[TEST 14] CRC dla 0x99
 CRC = 0x0000EC45

# |
```

Rys. 19. Testy przeprowadzone za pomocą main.c

Wszystkie testy uzyskały poprawne wyniki, są one większościowo odzwierciedleniem testów robionych w Verilogu, stąd łatwo sprawdzić poprawność wyników.